

FARO — Seguridad, protocolos y manejo de datos

Libro 1 de 3 · Para la institución, el área de TI y el jurado

Versión: septiembre 2026 · Reto SHIFT 2026 · Gamificación

Para qué sirve este libro

Una universidad que conecta una herramienta a su Canvas está prestando algo valioso: el acceso a datos académicos de sus estudiantes. Este libro explica, sin adornos, qué hace FARO con ese acceso, qué protocolos usa, por qué esos protocolos son seguros y qué garantías se pueden **verificar** en lugar de creer.

Cada afirmación de este libro tiene una de estas tres marcas:

- **[código]** — está escrito en el código del repositorio y se cita el archivo.
- **[prueba]** — existe una prueba automática que lo verifica (`server/test/e2e.ts`).
- **[pendiente]** — es el diseño para producción y todavía no está implementado.

Si algo no tiene marca, es contexto.

Índice

1. Resumen ejecutivo
2. Arquitectura de confianza: dónde vive cada dato
3. Protocolos: por qué son seguros
4. El token de Canvas: ciclo de vida
5. Minimización y retención de datos
6. Modelo de amenazas y controles
7. Cumplimiento: LFPDPPP y políticas institucionales
8. Preguntas frecuentes para la universidad
9. Lista de verificación antes de un piloto

Las cinco garantías, en una página

Garantía	Cómo se cumple	Marca
FARO solo lee. No puede modificar calificaciones, entregas, fechas ni contenido.	Todas las llamadas a Canvas son <code>GET</code> . El backend rechaza cualquier otro método antes de tocar la red. Los permisos LTI de escritura (<code>score</code> , <code>lineitem</code>) se rechazan explícitamente.	[código] [prueba]
Las credenciales nunca están en el navegador.	El token de Canvas y la clave de Gemini viven en <code>server/.env</code> y en la memoria del backend. La extensión no tiene ningún campo donde ponerlos y no tiene permiso de red hacia <code>instructure.com</code> .	[código] [prueba]
Solo seis rutas de Canvas, y nada más.	Una lista blanca en el backend (<code>server/src/allowlist.ts</code>) espejada en la extensión. Una ruta fuera de la lista se rechaza localmente, sin llamada a Canvas.	[código] [prueba]
El mentor de IA ve once campos y ningún dato personal.	Sin <code>GEMINI_API_KEY</code> en el backend (o con <code>VITE_MENTOR_MODE=local</code>), el mentor es local. Si responde Gemini, recibe, a través del backend, el contexto de once campos, el mensaje y los últimos 8 turnos. El contexto se construye en un solo archivo (<code>src/lib/mentorContext.ts</code>) que no tiene acceso al nombre, correo, foto ni calificaciones, y el backend lo vuelve a filtrar (<code>server/src/mentor.ts</code>).	[código] [prueba]
Nunca se muestra un puntaje de riesgo al estudiante.	Los estados de fricción existen solo para decidir la intervención; la interfaz usa lenguaje de apoyo.	[código]

1. Resumen ejecutivo

Qué es FARO, en términos de datos

FARO es un acompañante de persistencia para estudiantes adultos que estudian en línea. Lee de Canvas la estructura del curso y el avance del estudiante, detecta cuándo alguien se está desconectando y le ofrece una ruta de regreso. No enseña el curso, no califica, no sustituye a Canvas.

Desde el punto de vista de la institución, FARO es **un lector de solo lectura** de una parte pequeña y bien definida de Canvas, con una capa opcional de inteligencia artificial (Google Gemini, a través del backend) que recibe un contexto mínimo y sin identidad.

Las tres piezas y quién habla con quién



- **Canvas** sigue siendo la fuente de verdad académica. FARO no escribe ahí.
- **El backend** es el único proceso que conoce la credencial de Canvas y, si el mentor con IA está activo, la clave de la API de Gemini. Es un servidor Node.js sin dependencias de terceros (`server/`), lo que reduce a cero el riesgo de cadena de suministro en el proceso que sostiene los secretos.
- **La extensión** es solo interfaz. No tiene token, no tiene permiso de red hacia Canvas, y solo puede pedir al backend rutas que el backend ya decidió permitir.
- **El mentor** responde con Gemini solo si el backend tiene `GEMINI_API_KEY` ; si no, responde el mentor local, en el dispositivo, sin modelo de IA. Con el modo por omisión (`VITE_MENTOR_MODE=auto`), la extensión pregunta al backend (`GET /health` , sin datos del estudiante) si tiene mentor configurado. Una institución que no apruebe el mentor con IA construye con `VITE_MENTOR_MODE=local` o no le da clave al servidor. Cuando responde Gemini, las conversaciones abiertas van **a través del backend, nunca directo desde el navegador**, con un contexto de once campos (ninguno identifica a la persona), el mensaje del estudiante y los últimos 8 turnos de la conversación. Si Gemini no responde, contesta el mentor local.

Postura de seguridad en una frase

Reducir lo que FARO *puede* hacer hasta que coincida con lo que FARO *necesita* hacer, y hacer que esa reducción sea verificable por un administrador sin confiar en nosotros.

Estado actual, con honestidad

Componente	Estado
Backend con token, lista blanca, CORS restringido, límite de tasa, sin fuga de token en logs	Implementado y probado (38 verificaciones automáticas en total)
Extensión sin credenciales, modo <code>http</code> contra el backend	Implementado
Contrato de datos del mentor (11 campos), revalidado en el backend	Implementado
Mentor con Gemini (<code>POST /api/mentor</code>): clave solo en el servidor, conversación fuera de los logs, respaldo al mentor local	Implementado y probado contra un Gemini simulado ; no se ha ejercitado contra la API real de Google desde el entorno de desarrollo; el equipo lo valida con su propia clave
Pruebas contra un Canvas real (cuenta gratuita de Instructure)	Listo para ejecutar ; requiere un token del piloto
Lanzamiento LTI 1.3 firmado (OIDC + JWT)	Diseñado y documentado , rutas presentes que responden 501; requiere que la institución registre la herramienta
Token por estudiante vía OAuth2 (en vez de un token compartido)	Diseñado ; depende del registro LTI
Base de datos PostgreSQL con cifrado en reposo	Pendiente ; hoy el estado de FARO vive en el navegador del estudiante
Revisión de privacidad con la institución	Pendiente ; este libro es el insumo

2. Arquitectura de confianza: dónde vive cada dato

La pregunta que más importa en una revisión de seguridad no es "¿qué protocolo usan?" sino "¿dónde está cada dato y quién puede llegar a él?". Esta es la respuesta completa.

2.1 Zonas de confianza

FARO tiene cuatro zonas. Cada dato vive en exactamente una, y cruza una frontera solo por una razón documentada.

Zona	Qué contiene	Quién tiene acceso	Confianza
Z1 · Canvas	Todo el expediente académico	La institución	Máxima. FARO no la controla ni la altera.
Z2 · Backend FARO	El token de Canvas y, si el mentor con IA está activo, la clave de Gemini (ambos en memoria), la configuración, un caché breve del lanzamiento	El operador del backend	Alta. Es el único proceso con credenciales. No guarda conversaciones.
Z3 · Navegador del estudiante	Propósito, preferencias, sesiones registradas por FARO, perfil opcional, un instantáneo del curso mientras la pestaña está abierta	El estudiante	Media. Es su propio dispositivo.
Z4 · Servicio de IA (Google Gemini)	Por cada mensaje abierto al mentor: once campos de contexto, el mensaje y los últimos 8 turnos de la conversación	Google, bajo los términos de la API de Gemini (capítulo 7)	Activa solo si el backend tiene <code>GEMINI_API_KEY</code> y la extensión no se construyó con <code>VITE_MENTOR_MODE=local</code> . Se llega solo a través del backend. Nunca recibe identidad.

2.2 Inventario de datos

Cada fila responde: qué es, de dónde sale, dónde se guarda, quién lo ve, cuánto dura.

Datos que vienen de Canvas (Z1 → Z2 → Z3)

Dato	Ruta de Canvas	Se guarda en	Retención
Nombre, código y fecha de cierre del curso	<code>GET /api/v1/courses/:id</code>	Memoria del navegador mientras FARO está abierto	Se descarta al cerrar

Dato	Ruta de Canvas	Se guarda en	Retención
Módulos, sus ítems y el estado de completitud del estudiante	<code>GET /api/v1/courses/:id/modules?include[]=items</code>	Igual	Igual
Actividades con la entrega del estudiante (fecha, estado, puntaje)	<code>GET /api/v1/courses/:id/assignments?include[]=submission</code>	Igual	Igual
Vistas de página por hora y participaciones	<code>GET /api/v1/courses/:id/analytics/users/:uid/activity</code>	Igual. Solo se conservan las marcas de tiempo ; el conteo de vistas se descarta en el mapeo.	Igual
Cursos activos del estudiante	<code>GET /api/v1/courses?enrollment_state=active</code>	Backend, solo para elegir el curso cuando no está configurado	Un proceso
Id numérico del dueño del token	<code>GET /api/v1/users/self</code>	Backend, en caché. Nombre y avatar de esa respuesta se descartan.	Un proceso

[código] `src/data/canvas/endpoints.ts`, `src/data/canvas/client.ts` (función `mapActivity`), `server/src/routes/launch.ts`.

Datos que FARO crea (Z3)

Dato	Por qué existe	Se guarda en	Sale del navegador
Propósito (meta + frase de destino + minutos/día)	Personalizar la ruta y el mentor	<code>chrome.storage.local</code>	Meta, frase y minutos forman parte del contexto del mentor. Nada más.
Sesiones de estudio registradas por FARO	Impulso, logros, puntos	<code>chrome.storage.local</code>	No

Dato	Por qué existe	Se guarda en	Sale del navegador
Tarjetas de reconexión terminadas (fecha, ids de preguntas, aciertos a la primera)	Puntos y una oferta por día	<code>chrome.storage.local</code>	No. Las respuestas individuales no se guardan.
Canje de recompensa (nivel, fecha)	Un canje por semestre	<code>chrome.storage.local</code>	No en el prototipo (simulado). En producción, a la institución para entregar la recompensa.
Estado "pasó algo en mi vida"	Elegir la intervención	Memoria	No
Plan aceptado en el mentor (días, minutos, ids y títulos de los pasos)	Mostrarlo en Inicio	<code>chrome.storage.local</code> (clave <code>weekPlan</code>)	No. Se calcula localmente; nunca pasa por el modelo.
Estilo del mentor, idioma, tema	Preferencias	<code>chrome.storage.local</code>	Estilo e idioma forman parte del contexto del mentor
Nombre, foto (reducida), bio	Perfil opcional en Comunidad	<code>chrome.storage.local</code>	No al mentor. En producción, a otros estudiantes del curso si el estudiante lo activa.
Conversación con el mentor	La sesión actual	Solo memoria de la extensión; el backend no la guarda	Con el mentor local, no. Cuando responde Gemini, cada mensaje abierto va a Gemini a través del backend, con el contexto de once campos y los últimos 8 turnos de la conversación actual. El registro de ánimo, el plan, los puntos y las lecciones sin Gemini se resuelven localmente y no se envían al modelo cuando ocurren; sus líneas quedan en la conversación, así que pueden estar entre los 8 turnos que acompañan un mensaje abierto posterior.

[código] `src/state/storage.ts`, `src/state/store.tsx`, `src/types/index.ts` (comentarios de `Profile`).

Datos que nunca se recogen

Esta lista es tan importante como las anteriores. FARO **no** pide, lee ni almacena:

- Contraseñas de Canvas ni de la institución.
- Correo electrónico del estudiante.
- Lista de compañeros con nombre (no se llama al endpoint de inscripciones; **[prueba]** la ruta `/enrollments` se rechaza).
- Contenido de las entregas (textos, archivos, respuestas de exámenes).

- Calificaciones de otros estudiantes.
- Ubicación, dirección IP del estudiante (el backend registra solo la del cliente para el límite de tasa, en memoria, sin persistir).
- Historial de navegación fuera de las rutas de Canvas listadas.

2.3 Las fronteras y qué las cruza

Z1 Canvas → Z2 Backend	token del backend; respuesta completa de las 6 rutas
Z2 Backend → Z3 Navegador	JSON de Canvas ya paginado; NUNCA el token; NUNCA cabeceras de Canvas
Z3 Navegador → Z2 Backend	la ruta pedida (sin credencial); el backend la valida contra la lista blar
Z3 Navegador → Z2 → Z4 IA	[si se activa] 11 campos + el mensaje + últimos 8 turnos; NUNCA nombre, cc
Z2 Backend → Z4 IA	la clave de Gemini, solo en la cabecera x-goog-api-key, nunca en la URL

Lo que no aparece en el diagrama tampoco ocurre: la extensión no habla con Canvas (no tiene `host_permissions` para `instructure.com` [código] `public/manifest.json`), y la extensión tampoco habla con Google: cuando responde Gemini, `HttpMentorService` llama a `POST /api/mentor` en el backend, y es el backend el que llama a Gemini con la clave que solo él tiene [código] `src/services/mentor.ts`, `server/src/mentor.ts`. El backend reconstruye la petición desde cero (`sanitizeMentorRequest`): solo los once campos, con valores acotados; cualquier campo extra se descarta [prueba] un nombre, un correo, un id y una calificación añadidos al contexto nunca llegan a Gemini. Con el modo por omisión (`VITE_MENTOR_MODE=auto`), la extensión primero pregunta `GET /health` al backend (2 s de espera máxima, resultado guardado 30 s, sin ningún dato del estudiante) y solo usa Gemini si el servidor informa un mentor configurado [código] `probeGemini`, `AutoMentorService`. Con `VITE_MENTOR_MODE=local`, o si el servidor no tiene `GEMINI_API_KEY` o no responde, **ningún dato sale hacia un servicio de IA**: responde `LocalMentorService`, en el dispositivo, sin clave de API. En modo `local` ni siquiera se hace esa consulta a `/health` para el mentor.

2.4 Por qué el backend no tiene dependencias

El proceso que sostiene la credencial institucional (y la clave de Gemini) usa únicamente módulos nativos de Node.js (`node:http`, `node:crypto`, `node:fs`). No hay `node_modules` en producción. Esto no es minimalismo estético: cada paquete de terceros que corre dentro del proceso del token es código que la institución no escribió y que podría, en una actualización comprometida, leer el token de la memoria. La llamada a Gemini también es un `fetch` nativo, sin el SDK de Google. Cero paquetes es cero superficie de ese tipo.

[código] `server/package.json` (`"dependencies": {}`).

3. Protocolos: por qué son seguros

FARO no inventa ningún mecanismo de seguridad. Usa los mismos estándares que Canvas exige a cualquier integración y los aplica de la forma más restrictiva que el producto permite. Este capítulo explica cada uno: qué problema resuelve, cómo lo usa FARO y qué garantiza.

3.1 TLS (HTTPS) — el canal

Problema que resuelve: que nadie en la red (Wi-Fi de una cafetería, un proveedor de internet, un proxy institucional mal configurado) pueda leer o alterar lo que viaja entre FARO y Canvas.

Cómo lo usa FARO: el backend se niega a arrancar si `CANVAS_API_URL` no empieza con `https://`. La única excepción es `localhost`, y existe solo para las pruebas automáticas contra el Canvas simulado.

[código] `server/src/env.ts`, función `loadEnv`.

Garantía: el token de Canvas solo viaja cifrado. Canvas, por su parte, solo acepta la API por HTTPS.

3.2 Token portador (Bearer) — la credencial de la API

Problema que resuelve: demostrar a Canvas quién hace la petición sin enviar una contraseña.

Cómo lo usa FARO: el backend añade `Authorization: Bearer <token>` a cada petición hacia Canvas, que es el método que Instructure recomienda ("If possible, using the HTTP Authorization header is recommended"). El token nunca va en la URL, nunca en el cuerpo, nunca en un log.

[código] `server/src/canvas.ts`, función `canvasGet`. **[prueba]** "the token was sent to Canvas on every call, and only there" y "log lines carry no query strings".

Garantía: Instructure describe los tokens como "password equivalent". FARO los trata exactamente así: viven en un solo proceso, en memoria, y se leen de un archivo que git ignora. El capítulo 4 detalla su ciclo de vida.

Limitación del piloto, declarada: un token personal generado a mano es lo que Instructure indica para pruebas ("For testing your application before you've implemented OAuth, the simplest option is to generate an access token on your user's profile page"). Para múltiples usuarios, la propia documentación es tajante: "Applications in use by multiple users MUST use OAuth to obtain tokens". FARO cumple eso con el camino LTI + OAuth2 de la sección 3.4. **[pendiente]**

3.3 Lista blanca de rutas — el alcance

Problema que resuelve: que un token con más permisos de los necesarios (un token personal de estudiante puede leer todo lo que el estudiante ve en Canvas) se use solo para lo que FARO necesita.

Cómo lo usa FARO: el backend compara cada ruta pedida contra seis expresiones regulares. Solo `GET`. Solo los parámetros `include[]`, `per_page`, `enrollment_state` y `page`, con valores validados. Cualquier otra cosa se rechaza con `403 antes de tocar la red`. La ruta se decodifica antes de compararla, así que `%2e%2e` no puede escapar del patrón.

[código] `server/src/allowlist.ts`. **[prueba]** "a path outside the allow list is refused locally (no Canvas call)", "an unknown query parameter is refused", "path traversal is refused", "a write is refused even on an allowed path".

Garantía: aunque la extensión fuera modificada por alguien, no puede convertir el backend en un proxy general hacia Canvas. Y aunque el token del piloto pudiera leer la lista de compañeros, FARO **no puede pedirla**.

3.4 LTI 1.3 + OpenID Connect + JWT — el lanzamiento institucional

Este es el camino de producción. Está diseñado y documentado en el código; las rutas existen y responden `501` hasta que la institución registre la herramienta. **[pendiente]**

Problema que resuelve: saber con certeza *qué curso* y *qué persona* están abriendo FARO, sin pedirle a nadie que se identifique y sin que FARO tenga que confiar en lo que la extensión dice.

Cómo funciona (IMS Security Framework 1.0):

1. Canvas inicia un *login* OpenID Connect hacia `GET /lti/login`. FARO guarda un `state` y un `nonce` de vida corta y redirige al endpoint de autorización de Canvas.
2. Canvas responde con `POST /lti/launch` y un `id_token`: un JWT **firmado por Canvas** con RS256.
3. FARO descarga las claves públicas de Canvas (JWKS), verifica la firma, el emisor (`iss`), la audiencia (`aud` = el `client_id` de FARO), la expiración y que el `nonce` sea el emitido y no se haya usado.
4. De las *claims* verificadas, FARO lee exactamente dos cosas: el curso (`https://purl.imsglobal.org/spec/lti/claim/context`) y la persona (`sub`).
5. FARO emite su propia sesión (cookie `HttpOnly`, `Secure`, `SameSite=None` porque corre dentro de un `iframe` de Canvas) ligada a ese curso y esa persona.

[código] `server/src/routes/lti.ts` (especificación completa en los comentarios).

Garantía: la identidad del estudiante la afirma Canvas criptográficamente, no la extensión. FARO no puede ser engañado para mostrar el curso de otra persona porque no acepta ninguna afirmación de identidad que no venga firmada por la institución.

Permisos LTI: FARO solicita `contextmembership.readonly` (NRPS, para conteos de presencia agregada) y `result.readonly` (AGS, resultados de solo lectura). **Rechaza** `score` y `lineitem`, los dos permisos que permitirían escribir en el libro de calificaciones. Un administrador puede verificar esto en la pantalla de la clave de desarrollador sin leer una línea de código.

[código] `src/data/canvas/endpoints.ts`, constante `ltiScopes`.

3.5 OAuth2 con clave de desarrollador — el token por estudiante

[pendiente] Una vez registrada la herramienta, cada estudiante obtiene su propio token de acceso mediante el flujo OAuth2 de Canvas, con los *scopes* limitados a las seis rutas (en el formato `url:GET|/api/v1/...` que Canvas usa en la clave de desarrollador). Los tokens de este flujo expiran en una hora y se renuevan con un *refresh token*, ambos guardados solo en el backend. Se revocan con `DELETE /login/oauth2/token`.

Garantía: desaparece el token compartido del piloto. Cada estudiante ve exactamente lo que Canvas le deja ver a él, y revocar el acceso de FARO a una persona es una operación de un solo token.

[código] los *scopes* ya están escritos en `src/data/canvas/endpoints.ts`, constante `requiredScopes`.

3.6 CORS — quién puede llamar al backend desde un navegador

Problema que resuelve: que una página web cualquiera, abierta en el mismo navegador, use el backend de FARO como si fuera la extensión.

Cómo lo usa FARO: el backend mantiene una lista de orígenes permitidos (`FARO_ALLOWED_ORIGINS`). Un origen fuera de la lista recibe `403` sin datos ni cabeceras CORS. Los métodos anunciados son `GET`, `POST`, `OPTIONS`: `POST` existe para `POST /api/mentor` (y para el lanzamiento LTI, que hoy responde 501). Hacia Canvas el backend sigue enviando solo `GET`.

Excepción para desarrollo local: cuando el backend escucha en *loopback* (`FARO_HOST = 127.0.0.1`, `localhost` o `::1`, el valor por defecto), también acepta cualquier origen de extensión de Chrome con la forma `chrome-extension://<32 letras a-p>`, para que la extensión cargada sin empaquetar funcione sin copiar su id en `FARO_ALLOWED_ORIGINS`. Se apaga con `FARO_STRICT_ORIGINS=true`, y se apaga sola cuando el servidor escucha en otra interfaz (por ejemplo `0.0.0.0`). Al arrancar, el backend lo anuncia: *"Accepting the unpacked FARO extension (any chrome-extension:// origin, loopback only)"*. **En producción:** listar el id exacto de la extensión y fijar `FARO_STRICT_ORIGINS=true`.

[código] `server/src/http.ts`, método `corsHeaders`; `server/src/env.ts` (`allowExtensionOrigins`).

[prueba] "a browser origin outside the list gets 403 and no data", "CORS preflight succeeds for an allowed origin", "an extension origin is refused unless extension origins are enabled", "mentor: the unpacked extension can call it when extension origins are on (loopback dev)".

3.7 Límite de tasa — protección contra bucles

Problema que resuelve: una extensión con un error que reintenta en bucle podría enviar miles de peticiones a Canvas con el token institucional, y Canvas podría suspender ese token.

Cómo lo usa FARO: contador por cliente en ventana de un minuto (`FARO_RATE_LIMIT_PER_MINUTE`, 120 por defecto). Al superarlo, `429` con `Retry-After`. Las llamadas al modelo tienen además su propio tope por cliente (`FARO_MENTOR_PER_MINUTE`, 20 por defecto), que responde `429 mentor_rate_limited` antes de llamar a Gemini, para contener el costo.

[código] `server/src/http.ts`, clase `RateLimiter`; `server/src/routes/mentor.ts`. **[prueba]** "rate limit trips after the configured burst", "mentor: per-client cap answers 429 before calling the model".

3.8 Cabeceras de respuesta

Toda respuesta del backend lleva `Cache-Control: no-store` (ningún dato académico queda en cachés intermedios), `X-Content-Type-Options: nosniff` y `Referrer-Policy: no-referrer`.

[código] `server/src/http.ts`, función `send`. **[prueba]** "responses carry no-store and nosniff".

3.9 Resumen: qué protocolo protege qué

Amenaza	Protocolo / control	Estado
Escucha en la red	TLS obligatorio	[código]
Robo de credencial desde el navegador	El navegador no tiene credencial	[código] [prueba]
Uso del token más allá de lo necesario	Lista blanca de 6 rutas, solo GET	[código] [prueba]
Suplantación de estudiante	LTI 1.3 con JWT firmado por Canvas	[pendiente]
Token compartido entre personas	OAuth2 por estudiante	[pendiente]
Página maliciosa usando el backend	CORS con lista de orígenes	[código] [prueba]
Bucle de peticiones	Límite de tasa	[código] [prueba]
Escritura en el expediente	Sin permisos de escritura; scopes <code>score / lineitem</code> rechazados	[código]

4. El token de Canvas: ciclo de vida

El token es el dato más sensible que FARO maneja. Este capítulo sigue su vida completa: dónde nace, dónde vive, por dónde viaja, dónde **no** está nunca, y cómo muere. La sección 4.8 aplica las mismas reglas al segundo secreto del backend: la clave de la API de Gemini.

4.1 Nacimiento

Piloto (hoy): lo genera la persona dueña de la cuenta de Canvas, en *Cuenta* → *Configuración* → *Integraciones aprobadas* → *Nuevo token de acceso*. Canvas lo muestra una sola vez. Dos detalles de Canvas que juegan a favor:

- Las instituciones pueden **exigir fecha de expiración** para tokens de estudiantes (Instructure documenta un valor por defecto de 30 días para estudiantes y 90 para docentes), y pueden desactivar por completo la generación manual de tokens.
- El token hereda exactamente los permisos del usuario que lo creó: un token de estudiante no puede ver más que ese estudiante.

Producción: lo emite Canvas mediante OAuth2 a partir de una clave de desarrollador registrada por la institución, con *scopes* limitados y expiración de una hora. **[pendiente]**

4.2 Residencia

El token vive en **dos lugares y solo dos**:

1. `server/.env`, un archivo en la máquina que corre el backend. Está en `.gitignore` **[código]** y el repositorio incluye `server/.env.example` con el campo vacío para que nadie tenga que adivinar el formato.
2. La memoria del proceso del backend, en el objeto `Env`, desde el arranque hasta el apagado.

4.3 Viaje

Sale del backend en una única dirección: hacia Canvas, en la cabecera `Authorization`, por HTTPS, y solo para rutas de la lista blanca. Antes de seguir un enlace de paginación (`Link: rel="next"`), el backend verifica que la URL siga apuntando al host configurado de Canvas; un enlace que apuntara a otro dominio se rechaza, porque seguirlo enviaría el token fuera.

[código] `server/src/canvas.ts`, comprobación `url.startsWith(env.canvasApiUrl + '/')`.

4.4 Dónde no está nunca

Lugar	Por qué no	Verificación
La extensión	<code>CanvasConfig</code> no tiene campo para un token. No existe la variable.	<code>src/data/canvas/config.ts</code>

Lugar	Por qué no	Verificación
El build de Vite	Vite solo expone variables <code>VITE_*</code> ; el token no lleva ese prefijo y no está en <code>.env</code> de la raíz.	<code>.env.example</code> de la raíz lo dice explícitamente
Los logs del backend	El logger recibe método, ruta sin query, estado, duración e id. Nunca cabeceras.	[prueba] "the token was sent to Canvas on every call, and only there"
Las respuestas del backend	<code>describe(env)</code> es la única representación de la configuración que sale, y contiene <code>tokenPresent: true/false</code> , no el token.	[prueba] "health names the Canvas host and never the token"
Los mensajes de error	<code>CanvasError</code> guarda estado y ruta, nunca el cuerpo de la respuesta de Canvas.	<code>server/src/canvas.ts</code>
El repositorio git	<code>.gitignore</code> excluye <code>.env</code> en cualquier carpeta.	<code>.gitignore</code>
El servicio de IA	El contexto del mentor no tiene ningún campo de credencial, y la llamada a Gemini solo lleva la clave de Gemini.	<code>src/lib/mentorContext.ts</code> , <code>server/src/mentor.ts</code>

4.5 Rotación

Cambiar el token es editar `server/.env` y reiniciar el backend. No hay caché del token en ningún otro lugar que sobreviva al reinicio. Recomendación para el piloto: tokens con expiración de 30 días, rotados al vencer.

4.6 Revocación

- **Desde Canvas:** la persona dueña del token lo elimina en *Integraciones aprobadas*. Desde ese instante el backend recibe `401` y lo reporta como `canvas_token_rejected` sin intentar nada más. **[prueba]** "a wrong token is reported as rejected, not as a generic failure".
- **Desde el backend:** borrar el valor en `.env` y reiniciar. El backend se niega a arrancar con media configuración de Canvas (URL sin token o token sin URL) y se niega a arrancar si no tiene nada que servir (ni Canvas ni mentor). Si solo queda la clave de Gemini, arranca en modo "solo mentor" y las rutas de Canvas responden `503 canvas_not_configured`; `/health` lo muestra con `canvas: null`. Así no existe un estado "arrancó sin credencial y nadie se dio cuenta". **[prueba]** "config: half a Canvas setup is refused, mentor-only is accepted".
- **Producción (OAuth2):** `DELETE /login/oauth2/token` revoca el token del estudiante en Canvas. **[pendiente]**

4.7 Si el token se filtrara

Escenario: alguien obtiene acceso a `server/.env`.

1. **Alcance del daño:** lectura de lo que ese usuario de Canvas puede leer. No escritura: el token de un estudiante no puede alterar su expediente, y el de un docente tampoco a través de FARO, aunque sí

directamente contra Canvas. Por eso el piloto usa **tokens de cuentas de estudiante**, nunca de docente ni de administrador.

2. **Contención:** revocar el token en Canvas (un clic). Generar uno nuevo. Reiniciar el backend.
3. **Detección:** Canvas registra el uso de cada token; un patrón de uso desde una IP desconocida es visible para la institución.
4. **Prevención estructural:** el camino LTI + OAuth2 elimina el archivo `.env` con token de larga vida y lo sustituye por tokens de una hora emitidos por estudiante.

4.8 El segundo secreto: la clave de Gemini

Si la institución activa el mentor con IA, el backend guarda un segundo secreto: `GEMINI_API_KEY`. Sigue exactamente las reglas del token de Canvas.

- **Nacimiento:** se crea en Google AI Studio, en un proyecto de Google Cloud **con facturación activa** (capítulo 7 explica por qué el nivel gratuito no sirve con estudiantes reales).
- **Residencia:** `server/.env` y la memoria del proceso del backend. Nunca en la extensión: el `.env.example` de la raíz lo dice explícitamente, y `HttpMentorService` no tiene dónde guardarla. Es opcional: vacía, el mentor con IA está apagado y la extensión usa su mentor local.
- **Viaje:** sale solo hacia Google, en la cabecera `x-goog-api-key` de la llamada a `models/{modelo}:generateContent`, **nunca en la URL** (las URLs quedan en logs de proxies). **[prueba]** "mentor: the key goes in a header, never in the URL".
- **Dónde no está nunca:** en los logs ni en las respuestas del backend. `describe(env)` muestra solo `mentor: {provider, model}`. Si Google rechaza la clave (401/403), el backend responde `502 mentor_key_rejected` sin repetir el cuerpo de error de Google. **[prueba]** "mentor: a rejected key and a safety block map to fallback codes", "mentor: the key and the conversation never appear in the log".
- **Rotación y revocación:** editar `server/.env` y reiniciar. Si la clave se expone, revocarla en Google AI Studio / Google Cloud y generar otra.
- **Alcance del daño si se filtra:** alguien podría hacer llamadas a Gemini a cargo del proyecto de la institución. No da acceso a Canvas ni a datos de estudiantes: FARO no guarda conversaciones. `FARO_MENTOR_PER_MINUTE` (20 por omisión) limita por cliente cuántas llamadas al modelo acepta el backend, para contener el costo.

5. Minimización y retención de datos

El principio rector de FARO en materia de datos es simple: **no recoger un dato porque es técnicamente posible, sino porque una función concreta lo necesita**. Este capítulo muestra cómo se aplica en cada capa, y qué pasa con los datos con el tiempo.

5.1 Minimización en la capa de Canvas

Canvas devuelve mucho más de lo que FARO usa. La capa de integración descarta lo que no necesita **en el momento de leerlo**, no después.

Lo que Canvas envía	Lo que FARO conserva	Lo que se descarta
Objeto de curso (decenas de campos)	id, nombre, código, fechas de inicio y fin	Todo lo demás: configuración, permisos, imagen, etc.
Módulos con ítems	id, nombre, posición, estado, conteo de ítems, <code>content_id</code> de cada ítem y si está completado	Títulos de páginas, URLs externas, archivos
Actividades con entrega	id, nombre, fecha límite, puntos posibles, y de la entrega: fecha, estado, puntaje	Descripción de la actividad, rúbrica, adjuntos, comentarios del docente
Actividad del estudiante	Solo marcas de tiempo. El número de vistas por hora se elimina.	Conteos de vistas, URLs visitadas
<code>users/self</code>	El id numérico	Nombre, avatar, correo, zona horaria

[código] `src/data/canvas/raw.ts` declara solo los campos leídos; `src/data/canvas/client.ts` hace el mapeo; `server/src/routes/launch.ts` conserva únicamente `self.data.id`.

Por qué importa el caso de la actividad: "cuántas páginas vio a las 3 de la tarde" es un dato de vigilancia. "Cuándo fue la última vez que entró" es el único dato que el motor de fricción necesita. FARO conserva el segundo y destruye el primero.

5.2 Minimización en el mentor de IA

El contexto que recibe el modelo se construye en una sola función y contiene exactamente estos once campos:

```
{
  "course": "Gestión de Proyectos",
  "progress": 44,
  "next_activity": "Caso de estudio: alcance",
  "estimated_time": 25,
  "student_goal": "career_growth",
  "destination": "Liderar mis propios proyectos",
  "available_time": 20,
  "momentum": 62,
  "friction_state": "FRICTION",
  "style": "encouraging",
  "language": "es"
}
```

No hay nombre, correo, id de Canvas, calificaciones, foto, historial de otras conversaciones ni clasificación de riesgo de abandono. El estado de fricción es una de cinco etiquetas de intervención (`FLOWING` , `FRICITION` , `POSSIBLE_OVERWHELM` , `DISCONNECTION` , `RECOVERY`), no un puntaje.

La única información de origen humano es la **frase de destino**, que el estudiante escribe libremente al iniciar. Se le muestra el ejemplo *"Liderar mis propios proyectos en vez de solo ejecutarlos"* para orientar hacia una meta, no hacia datos personales. En producción, esta frase debe pasar por una advertencia explícita: *"no incluyas datos que te identifiquen"*. **[pendiente]**

[código] `src/lib/mentorContext.ts` , `src/types/index.ts` (interfaz `MentorContext`).

Cuando el mentor usa Gemini (el backend tiene `GEMINI_API_KEY` y la extensión no se construyó con `VITE_MENTOR_MODE=local`), lo que sale hacia Google, siempre a través del backend y nunca directo desde el navegador, es: estos once campos, el mensaje del estudiante y los últimos 8 turnos de la conversación actual (rol y texto, nada más). El backend no confía en que la extensión haya hecho bien su parte: `sanitizeMentorRequest` reconstruye la petición con solo los once campos, comprueba los valores enumerados, recorta longitudes y descarta cualquier campo extra **[prueba]** "mentor: extra fields in the context never reach Gemini", "mentor: history is trimmed to the last 8 turns".

Lo estructurado nunca pasa por el modelo: el registro de ánimo ("Noté que algo cambió..."), las sesiones de Enfoque, el plan de fin de semana y las respuestas sobre puntos se calculan en la extensión con datos reales y no se envían al modelo. Sus líneas sí quedan en la conversación, así que pueden viajar como parte de los últimos 8 turnos si después el estudiante escribe un mensaje abierto. Una petición de "haz mi trabajo calificado" la contesta el propio backend sin llamar al modelo **[prueba]** "mentor: "do my homework" is refused before any model call".

[código] `server/src/mentor.ts` .

5.3 Minimización en Comunidad

Comunidad es la parte de FARO con más tentación de recoger datos sociales, y por eso tiene las reglas más estrictas, escritas en los tipos:

- **No existe campo de seguidores ni de ranking.** No es que estén ocultos: no están en el modelo de datos, así que ninguna función puede calcularlos.
- **La presencia es agregada.** "23 estudiantes estudiando ahora" es un conteo. "Andrea lleva 37 minutos" nunca aparece, y el tipo `CoursePresence` solo admite números.
- **Los compañeros no tienen foto.** Solo iniciales y un color decorativo. Únicamente el propio estudiante puede añadir una imagen a su avatar, reducida en su navegador antes de guardarse.
- **Las reacciones se muestran por separado**, nunca sumadas en un marcador.

[código] `src/types/index.ts` , comentarios de `CommunityAuthor` y `CoursePresence` .

5.4 Retención

Dato	Dónde	Cuánto dura	Cómo se borra
Instantáneo del curso (de Canvas)	Memoria del navegador	Mientras FARO está abierto	Al cerrar la pestaña o el panel

Dato	Dónde	Cuánto dura	Cómo se borra
Registro de peticiones a Canvas	Memoria del navegador, 30 entradas	Mientras FARO está abierto	Automático; nunca se persiste ni se envía
Propósito, preferencias, sesiones, perfil, plan aceptado en el mentor (<code>weekPlan</code>)	<code>chrome.storage.local</code>	Hasta que el estudiante lo borre	Botón <i>Restablecer</i> en Progreso, o desinstalar la extensión; el plan también con <i>Quitar plan</i> en Inicio
Conversación con el mentor	Memoria de la extensión	La sesión	Al cerrar FARO o con <i>Restablecer</i> . El backend no la guarda ni la registra
Lo enviado a Gemini (cuando responde Gemini)	Google	Según los términos de la API de Gemini (capítulo 7)	Con clave de pago, Google lo registra por tiempo limitado solo para detectar abuso y por requisitos legales
Token de Canvas	<code>server/.env</code> + memoria del backend	Hasta rotación o revocación	Editar <code>.env</code> y reiniciar; revocar en Canvas
Clave de Gemini	<code>server/.env</code> + memoria del backend	Hasta rotación o revocación	Editar <code>.env</code> y reiniciar; revocar en Google
Caché del lanzamiento (id de curso, id de usuario)	Memoria del backend	Hasta reiniciar el proceso	Reiniciar
Logs del backend	Salida estándar	Los define el operador	Los define el operador; no contienen datos personales, query strings, cuerpos de petición ni nada de la conversación con el mentor

Producción [pendiente]: cuando exista PostgreSQL, se propone:

- Sesiones de estudio, propósito y preferencias: mientras la inscripción esté activa, más 90 días para permitir "regresar" a un curso pausado; después, borrado automático.
- Conversaciones con el mentor: no se almacenan más allá de la sesión, salvo consentimiento explícito para mejorar el producto, en cuyo caso se anonimizan.
- Eventos de fricción e intervenciones: agregados para métricas del piloto, con el id del estudiante sustituido por un id pseudónimo que solo la institución puede resolver.

5.5 Lo que el estudiante puede ver y controlar

- **Perfil** → **Canvas** muestra en vivo cada llamada que FARO hace, con su ruta, estado y latencia. La transparencia no es un documento: es una pantalla.
- **Perfil** → **Canvas** muestra los permisos LTI que FARO rechaza.
- **FARO Mentor** dice, al pie, qué ve el mentor y quién responde: *"Responde Gemini, a través del servidor de FARO..."* o *"Mentor local: sin modelo de IA, nada sale de tu dispositivo."* Los mensajes escritos por Gemini llevan un anillo violeta en el avatar de FARO.
- **Progreso** → **Restablecer** borra todo el estado local de FARO.
- Nada del perfil es obligatorio. FARO funciona completo sin nombre ni foto.

6. Modelo de amenazas y controles

Un modelo de amenazas responde tres preguntas: qué queremos proteger, de quién, y qué hemos hecho al respecto. Esta tabla es la respuesta, amenaza por amenaza. La columna *Prueba* nombra la verificación automática en `server/test/e2e.ts` cuando existe.

6.1 Activos

- 1. **El token de Canvas** — da acceso de lectura al expediente del dueño.
- 2. **Los datos académicos del estudiante** — avance, entregas, fechas.
- 3. **La identidad del estudiante** — quién es la persona detrás de una sesión de FARO.
- 4. **La integridad del expediente** — que nada de lo que FARO hace pueda alterar Canvas.
- 5. **La confianza del estudiante** — que FARO no se convierta en vigilancia.
- 6. **La clave de Gemini** — si el mentor con IA está activo, permite hacer llamadas al modelo a cargo del proyecto de la institución.

6.2 Amenazas y controles

#	Amenaza	Activo	Control	Estado	Prueba
T1	Un atacante extrae la credencial del código de la extensión (las extensiones son código público)	Token	No hay credencial en la extensión. <code>CanvasConfig</code> no tiene campo para ella.	[código]	—
T2	Una extensión modificada usa el backend para leer cualquier ruta de Canvas	Datos	Lista blanca de 6 rutas en el backend; <code>403</code> local sin llamada a Canvas	[código]	"a path outside the allow list is refused locally"
T3	Una extensión modificada intenta escribir en Canvas a través del backend	Integridad	Solo <code>GET</code> ; <code>405</code> para otros métodos; sin scopes de escritura	[código]	"a write is refused even on an allowed path"
T4	Manipulación de parámetros para ampliar la respuesta de Canvas (<code>all_dates</code> , <code>include[]=user</code>)	Datos	Parámetros y valores validados; el resto se rechaza	[código]	"an unknown query parameter is refused"
T5	<i>Path traversal</i> codificado (<code>..%2F</code>) para salir del patrón	Datos	Decodificación antes de comparar; <code>..</code> y <code>//</code> rechazados	[código]	"path traversal is refused"

#	Amenaza	Activo	Control	Estado	Prueba
T6	Una página web maliciosa en el mismo navegador llama al backend	Datos	CORS con lista de orígenes; origen desconocido → 403 sin datos	[código]	"a browser origin outside the list gets 403"
T7	Bucle de peticiones agota la cuota de Canvas y expone el token a suspensión	Token	Límite de tasa por cliente, 429	[código]	"rate limit trips after the configured burst"
T8	El token aparece en logs, errores o respuestas	Token	Logger sin cabeceras ni query; errores sin cuerpo de Canvas; describe() sin token	[código]	"health names the Canvas host and never the token", "log lines carry no query strings"
T9	Un Link de paginación manipulado redirige el token a otro dominio	Token	El backend verifica que cada página siga en el host de Canvas configurado	[código]	—
T10	Escucha de red entre backend y Canvas	Token, datos	HTTPS obligatorio; el backend no arranca con http:// fuera de localhost	[código]	—
T11	Suplantación: alguien afirma ser otro estudiante	Identidad	Piloto: el backend deduce la identidad del token, no de la extensión. Producción: JWT firmado por Canvas (LTI 1.3)	[código] / [pendiente]	"launch resolves the token owner"
T12	Cachés intermedios conservan datos académicos	Datos	Cache-Control: no-store en toda respuesta	[código]	"responses carry no-store and nosniff"
T13	Un paquete de terceros comprometido lee el token de la memoria del backend	Token	Cero dependencias en el backend	[código]	—
T14	El servicio de IA recibe datos personales	Identidad, confianza	Contexto de 11 campos construido en una sola función; sin nombre, correo, ids, calificaciones. El backend lo reconstruye (sanitizeMentorRequest) y descarta cualquier campo extra; historial limitado a 8 turnos	[código]	"mentor: extra fields in the context never reach Gemini", "mentor: history is trimmed to the last 8 turns"
T15	FARO se convierte en herramienta de vigilancia (quién estudia cuánto)	Confianza	Presencia agregada en el modelo de datos; sin ranking ni seguidores; conteos de vistas descartados	[código]	—

#	Amenaza	Activo	Control	Estado	Prueba
T16	Se expone al estudiante una clasificación de riesgo de abandono	Confianza	Los estados de fricción son internos; la interfaz usa lenguaje de apoyo; no existe puntaje de riesgo	[código]	—
T17	Un token de docente o administrador en el piloto amplía el alcance de una fuga	Token, datos	Política: el piloto usa únicamente tokens de cuentas de estudiante	Política	—
T18	El backend arranca sin credencial y sirve datos vacíos como si fueran reales	Confianza	Se niega a arrancar con media configuración de Canvas o sin nada que servir; en modo "solo mentor", las rutas de Canvas responden <code>503 canvas_not_configured</code> ; con token inválido responde <code>canvas_token_rejected</code>	[código]	"a wrong token is reported as rejected", "mentor-only server: Canvas routes say canvas_not_configured", "config: half a Canvas setup is refused, mentor-only is accepted"
T19	Una cuenta de estudiante no puede leer analíticas y FARO inventa actividad	Confianza	<code>401 / 403</code> se pasa al cliente; FARO usa fechas de entregas y lo dice en pantalla	[código]	"Canvas 401 on analytics is passed through"
T20	Robo del archivo <code>server/.env</code>	Token, clave de Gemini	Alcance limitado a lectura de un estudiante; revocación de un clic en Canvas; revocación de la clave en Google; el camino OAuth2 elimina el token del archivo	Política / [pendiente]	—
T21	La clave de Gemini se filtra por la URL, los logs, una respuesta o un error	Clave de Gemini	Solo en la cabecera <code>x-goog-api-key</code> ; nunca en la URL; <code>describe()</code> muestra solo proveedor y modelo; el cuerpo de error de Google nunca se repite ni se registra	[código]	"mentor: the key goes in a header, never in the URL", "mentor: a rejected key and a safety block map to fallback codes", "mentor: the key and the conversation never appear in the log"
T22	Las conversaciones con el mentor quedan en los logs del backend	Identidad, confianza	El logger solo recibe método, ruta, estado, duración e id; nunca cuerpos. Ni mensaje, ni respuesta, ni contexto	[código]	"mentor: the key and the conversation never appear in the log"
T23	Un cliente dispara llamadas al modelo en bucle y genera costo	Clave de Gemini	Límite por cliente de llamadas al modelo (<code>FARO_MENTOR_PER_MINUTE</code> , 20) además del límite global; cuerpo máximo de 32 KB	[código]	"mentor: per-client cap answers 429 before calling the model", "mentor: a non-JSON body gets 415 and a huge body 413"

#	Amenaza	Activo	Control	Estado	Prueba
T24	El mentor hace el trabajo calificado del estudiante	Confianza, integridad académica	Las peticiones de "haz mi tarea" se responden en el backend sin llamar al modelo; el prompt de sistema prohíbe producir trabajo calificado	[código]	"mentor: "do my homework" is refused before any model call"
T25	Google usa las conversaciones para mejorar sus productos o las leen revisores humanos	Identidad, confianza	Política: solo claves de un proyecto con facturación activa (servicios de pago); nunca el nivel gratuito con estudiantes reales (capítulo 7)	Política	—
T26	Otra extensión instalada en el mismo navegador llama al backend local (en <i>loopback</i> se acepta cualquier origen <code>chrome-extension://</code>)	Datos, clave de Gemini	Solo cuando el servidor escucha en <i>loopback</i> ; se apaga con <code>FARO_STRICT_ORIGINS=true</code> y sola en cualquier otra interfaz; el tope por cliente del mentor sigue aplicando; en despliegue, id exacto de la extensión + <code>FARO_STRICT_ORIGINS=true</code>	[código] / Política	"an extension origin is refused unless extension origins are enabled", "config: half a Canvas setup is refused, mentor-only is accepted"

6.3 Lo que este modelo no cubre todavía

- **Autenticación de la extensión ante el backend en producción.** Hoy el backend confía en el origen (CORS) y en escuchar solo en `127.0.0.1`. En un despliegue institucional, la sesión emitida tras el lanzamiento LTI es lo que autoriza cada llamada. **[pendiente]**
- **Cifrado en reposo** de la futura base de datos. **[pendiente]**
- **Auditoría de accesos** por parte de la institución: quién consultó qué y cuándo. El backend registra ruta y estado por petición; falta la persistencia y el id pseudónimo. **[pendiente]**
- **Revisión de seguridad externa.** Este documento es autoevaluación; un piloto institucional debe incluir una revisión por el área de seguridad de la universidad.

7. Cumplimiento: LFPDPPP y políticas institucionales

Este capítulo es orientación técnica para preparar la revisión legal de la institución. No sustituye la opinión del área jurídica ni del responsable de protección de datos de la universidad.

7.1 El marco aplicable en México

La **Ley Federal de Protección de Datos Personales en Posesión de los Particulares (LFPDPPP)** fue publicada en el Diario Oficial de la Federación el 20 de marzo de 2025 y entró en vigor el 21 de marzo de 2025. Sustituye a la ley de 2010. La autoridad reguladora para el sector privado ya no es el INAI, disuelto, sino la **Secretaría Anticorrupción y Buen Gobierno**.

Tres puntos de la nueva ley que tocan directamente a FARO:

- 1. **Definición de dato personal.** Es toda información de una persona identificada *o identificable*, "cuando su identidad pueda determinarse directa o indirectamente a través de cualquier información". Un id numérico de Canvas es, por tanto, dato personal aunque no lleve nombre: es indirectamente identificable por la institución.
- 2. **Aviso de privacidad.** Debe identificar datos sensibles y distinguir las finalidades que requieren consentimiento expreso. Cuando los datos se recaban por medios electrónicos, debe ofrecerse un aviso simplificado que remita al completo.
- 3. **Consentimiento.** Puede ser expreso o tácito; es tácito cuando, puesto a disposición el aviso de privacidad, el titular no manifiesta lo contrario. Para datos sensibles sigue exigiéndose consentimiento expreso.

Nota sobre el sujeto obligado. Una universidad privada como Tecmilenio es un particular y le aplica la LFPDPPP. Una universidad pública se rige por la Ley General de Protección de Datos Personales en Posesión de Sujetos Obligados. Ambas comparten los principios; el análisis de este capítulo es válido para las dos, pero el instrumento y la autoridad cambian. **La institución debe confirmar cuál le aplica.**

7.2 Los principios, uno por uno

Principio	Qué exige	Cómo lo cumple FARO	Marca
Licitud	Tratar datos conforme a la ley y sin engaño	FARO solo lee lo que el estudiante ya puede ver de sí mismo en Canvas; el registro de peticiones es visible en pantalla	[código]
Consentimiento	Contar con la voluntad del titular	Piloto: consentimiento expreso e informado de cada participante (plantilla en 7.4). Producción: aviso de privacidad integrado en el lanzamiento LTI	Política / [pendiente]
Información	Aviso de privacidad accesible	Pantalla <i>Perfil</i> → <i>Canvas</i> y pie del Mentor muestran qué se consulta, qué se envía y quién responde (Gemini o el mentor local); aviso formal por redactar con la institución	[código] / [pendiente]

Principio	Qué exige	Cómo lo cumple FARO	Marca
Calidad	Datos exactos y actualizados	Canvas es la fuente de verdad; FARO no copia el expediente, lo lee en cada apertura	[código]
Finalidad	Usar los datos solo para lo informado	Cada ruta de Canvas tiene un propósito escrito junto a ella (<code>server/src/allowlist.ts</code> , campo <code>purpose</code>); no hay uso secundario	[código]
Lealtad	No obtener datos por medios engañosos	No hay rastreo fuera de las 6 rutas; no hay <i>dark patterns</i> para obtener el perfil (todo es opcional)	[código]
Proporcionalidad	Solo los datos necesarios	Capítulo 5: conteos de vistas descartados, nombre y avatar de <code>users/self</code> descartados, mentor con 11 campos revalidados en el backend e historial limitado a 8 turnos	[código] [prueba]
Responsabilidad	Velar por el cumplimiento y poder demostrarlo	Este libro; las 38 pruebas automáticas; la lista de verificación del capítulo 9	[prueba]

7.3 Datos sensibles

FARO **no trata datos sensibles** según la definición de la ley (origen étnico, salud, creencias, opiniones políticas, preferencia sexual, datos genéticos). Dos matices que la institución debe considerar:

- El estado *"pasó algo en mi vida"* (menos tiempo, abrumado, necesito una pausa) es autodeclarado, se usa solo para elegir una intervención y **no se persiste**. No es parte de los once campos del contexto. Si el estudiante lo elige en el registro de ánimo del mentor, esa elección queda como un turno de la conversación, y cuando responde Gemini puede viajar entre los últimos 8 turnos de un mensaje abierto posterior. No es un dato de salud, pero el aviso de privacidad debe mencionarlo para no dejar dudas.
- La **frase de destino** es texto libre. Si un estudiante escribiera ahí un dato sensible, FARO lo guardaría localmente y lo incluiría en el contexto del mentor (y, cuando responde Gemini, llegaría a Google). Mitigación: advertencia en pantalla **[pendiente]** y política de no almacenar conversaciones.
- Los **mensajes al mentor** también son texto libre. Cuando responde Gemini, van a Google a través del backend. Si un estudiante menciona una crisis o querer hacerse daño, las reglas del mentor le piden responder con cuidado, dejar de hablar de estudio y animarlo a contactar de inmediato al servicio de apoyo estudiantil de la universidad o a servicios de emergencia. FARO no es un servicio de apoyo psicológico; la institución debe decidir qué canal nombrar.

7.4 Consentimiento para el piloto

Para un piloto con cinco a treinta estudiantes, se recomienda consentimiento expreso por escrito. Elementos mínimos que debe contener:

1. Quién es el responsable (la institución) y quién el encargado (el equipo de FARO).
2. Qué datos se leen de Canvas (las seis rutas, en lenguaje llano).
3. Qué datos crea FARO (propósito, sesiones, preferencias, perfil opcional).
4. Que FARO no modifica nada en Canvas.

5. Si la institución activa Gemini: que el mentor puede responder con un modelo de Google; que recibe, a través del backend de FARO, un contexto sin nombre ni correo, el mensaje del estudiante y los últimos 8 turnos de la conversación; que FARO no guarda las conversaciones; y que la pantalla del mentor siempre dice quién responde. Que solo participan personas mayores de 18 años.
6. Cuánto tiempo se conservan los datos y cómo pedir su borrado (*Progreso* → *Restablecer*, y revocación del token).
7. Que la participación es voluntaria y retirarse no afecta la calificación.
8. Cómo ejercer derechos ARCO ante la institución.

7.5 Derechos ARCO

Derecho	Cómo se ejerce en FARO hoy	En producción
Acceso	<i>Perfil</i> muestra todo lo que FARO guarda del estudiante; <i>Perfil</i> → <i>Canvas</i> muestra todo lo que lee	Exportación desde el backend
Rectificación	Los datos académicos se corrigen en Canvas (fuente de verdad); los de FARO, en Perfil	Igual
Cancelación	<i>Progreso</i> → <i>Restablecer</i> borra el estado local; desinstalar la extensión lo elimina todo	Borrado en base de datos por solicitud
Oposición	No instalar la extensión; retirar el consentimiento del piloto	Desactivar la herramienta LTI para ese estudiante

7.6 Transferencias

- **A Canvas (Instructure):** FARO no transfiere nada a Canvas; solo lee. La relación de la institución con Instructure ya está cubierta por su contrato existente.
- **Al proveedor del modelo de IA (Google, API de Gemini):** solo si el backend tiene `GEMINI_API_KEY` (con el modo por omisión, `VITE_MENTOR_MODE=auto`, la extensión lo detecta sola; con `gemini`, siempre lo intenta). Una institución que no apruebe el mentor con IA construye con `VITE_MENTOR_MODE=local` o no le da clave al servidor. Es una transferencia de los 11 campos del contexto, el mensaje del estudiante y los últimos 8 turnos de la conversación actual; nunca nombre, correo, ids de Canvas, calificaciones ni foto. Debe constar en el aviso de privacidad. Las condiciones de uso de esos datos dependen del tipo de clave (sección 7.8). Con `VITE_MENTOR_MODE=local`, o sin clave en el servidor, no hay transferencia.
- **Entre dispositivos del estudiante [pendiente]:** cuando exista el backend con base de datos, el estado de FARO se sincronizará a través de él. Es tratamiento por el encargado, no transferencia a tercero.

7.7 Políticas institucionales de Canvas

Independientemente de la ley, la institución controla en Canvas:

- Si los estudiantes pueden generar tokens de acceso y con qué expiración.

- Qué *scopes* concede a la clave de desarrollador de FARO (recomendación: exactamente los de `requiredScopes`).
- Si la herramienta LTI se instala a nivel de cuenta, subcuenta o curso.
- Si los estudiantes pueden ver sus propias analíticas (afecta si FARO usa el endpoint de actividad o el respaldo por entregas).

Ninguna de estas decisiones requiere cambios en FARO; el backend respeta lo que Canvas permita y reporta lo que no.

7.8 Términos de la API de Gemini

Fuente: *Gemini API Additional Terms of Service*, ai.google.dev/gemini-api/terms . La institución debe revisar la versión vigente antes del piloto; lo que sigue es lo que FARO asume.

	Servicios de pago (proyecto de Google Cloud con facturación activa)	Servicios gratuitos (sin facturación)
¿Google usa los <i>prompts</i> y respuestas para mejorar sus productos?	No	Sí
¿Qué hace Google con ellos?	Los registra por tiempo limitado, solo para detectar abuso y por requisitos legales	Los usa para mejorar productos; revisores humanos pueden leerlos
Advertencia de los términos	—	No enviar información personal

Regla de FARO: con estudiantes reales, solo una clave de un proyecto con **facturación activa**. Nunca el nivel gratuito. El nivel gratuito sirve para que el equipo pruebe con datos inventados, nada más.

Edad. Los términos exigen que los usuarios de la API de Gemini sean mayores de 18 años. El piloto es con estudiantes adultos, mayores de 18; la institución debe confirmarlo al reclutar.

Lo que FARO añade por su cuenta, independiente de los términos de Google: la clave solo vive en el backend; el backend no guarda ni registra las conversaciones; el contexto se filtra dos veces (extensión y backend); y si Gemini falla, responde el mentor local, así que el servicio no depende de Google para seguir funcionando.

8. Preguntas frecuentes para la universidad

Las preguntas que un área de TI, un responsable de datos o un directivo académico hacen antes de conectar una herramienta a Canvas, con respuestas cortas y la referencia para comprobarlas.

Acceso y alcance

¿FARO puede modificar calificaciones, entregas o fechas? No. Todas las llamadas a Canvas son de lectura (`GET`). El backend rechaza cualquier otro método antes de contactar a Canvas, y los permisos LTI que permitirían escribir en el libro de calificaciones (`score` , `lineitem`) se rechazan explícitamente. Se comprueba en `server/src/allowlist.ts` y en la pantalla de la clave de desarrollador de Canvas.

¿Qué ve exactamente FARO de Canvas? Seis rutas: el curso, sus módulos con ítems, las actividades con la entrega del propio estudiante, la actividad (marcas de tiempo) del propio estudiante, la lista de cursos activos del estudiante y su id numérico. Nada más. La lista está en un archivo de treinta líneas: `server/src/allowlist.ts`.

¿Ve FARO los datos de otros estudiantes? No. No llama al endpoint de inscripciones ni al de usuarios del curso. En producción, el conteo de "cuántos están estudiando" vendrá del servicio NRPS de LTI en modo agregado, sin nombres.

¿Puede FARO ver el contenido de una entrega o de un examen? No. Lee fecha, estado y puntaje de la entrega, no su contenido. Los campos que se leen están declarados en `src/data/canvas/raw.ts`; lo que no está ahí, se ignora.

¿Un estudiante puede usar FARO para ver el curso de otro? No. En el piloto, el backend deduce la identidad del token, no de lo que la extensión afirma. En producción, la identidad la firma Canvas (LTI 1.3) y FARO no acepta otra.

Credenciales

¿Dónde está el token de Canvas? En `server/.env`, en la máquina que corre el backend, y en la memoria de ese proceso. En ningún otro lugar. La extensión no tiene ningún campo para un token y no tiene permiso de red hacia `instructure.com`.

¿El token está en el código fuente o en GitHub? No. `.env` está en `.gitignore`. El repositorio contiene `server/.env.example` con el campo vacío.

¿Dónde está la clave de Gemini? Igual que el token: en `server/.env` y en la memoria del backend, y solo si se quiere el mentor con IA. Nunca en la extensión. Viaja a Google solo en la cabecera `x-goog-api-key`, nunca en la URL, y no aparece en logs ni respuestas. Si se expone, se revoca en Google y se genera otra. Capítulo 4.8.

¿Qué pasa si el token se filtra? Se revoca en Canvas en un clic, se genera otro, se reinicia el backend. El alcance del daño es lectura de lo que ese estudiante ve de sí mismo. Por eso el piloto usa solo tokens de cuentas de estudiante, nunca de docente ni de administrador. Capítulo 4.

¿Cada estudiante necesita generar un token? En el piloto, sí, uno por participante (o un token de una cuenta de prueba). Es la práctica que Instructure indica para pruebas. Para un despliegue real, FARO usa

OAuth2 con clave de desarrollador y los tokens los emite Canvas automáticamente, con una hora de vida.

Inteligencia artificial

¿Qué modelo usa FARO? Google Gemini (`GEMINI_MODEL` , por omisión `gemini-3.5-flash` ; los nombres de modelo cambian y deben verificarse en `ai.google.dev/gemini-api/docs/models`), pero solo si el backend tiene `GEMINI_API_KEY` . Si no, ninguno: responde el mentor local, determinista, y nada sale del dispositivo. `VITE_MENTOR_MODE` tiene tres valores: `auto` (por omisión: la extensión pregunta a `GET /health` del backend, con 2 s de espera y resultado guardado 30 s, sin datos del estudiante, y usa Gemini si el servidor lo tiene), `local` (nunca sale del dispositivo, ni siquiera esa consulta) y `gemini` (siempre lo intenta). La pantalla del mentor siempre dice cuál de los dos responde, según esa comprobación en vivo.

¿Qué recibe el modelo de IA? Solo cuando responde Gemini, y siempre a través del backend: once campos (nombre del curso, porcentaje de avance, siguiente actividad y su duración, meta declarada, frase de destino, minutos disponibles, impulso, estado de fricción, estilo e idioma), el mensaje del estudiante y los últimos 8 turnos de la conversación actual. El contexto se arma en `src/lib/mentorContext.ts` y el backend lo vuelve a filtrar en `server/src/mentor.ts` . No recibe nombre, correo, ids de Canvas, calificaciones, foto ni conversaciones anteriores.

¿El modelo puede hacer la tarea del estudiante? No. Hay un filtro de peticiones del tipo "hazme el examen" que responde con una negativa y ofrece dividir la tarea, explicar el concepto o revisar el razonamiento; cuando Gemini está activo, el backend contesta esas peticiones sin llamar al modelo. Además, las reglas del prompt de sistema prohíben producir trabajo calificado. El filtro es igual en los cinco estilos de comunicación.

¿Hoy los datos van a un proveedor de IA? Solo si el backend tiene `GEMINI_API_KEY` y la extensión no se construyó con `VITE_MENTOR_MODE=local` . Sin clave, o con `local` , no: el mentor es local y determinista. Una institución que no apruebe el mentor con IA construye con `VITE_MENTOR_MODE=local` o no le da clave al servidor. Antes de usarlo con estudiantes reales hace falta el aviso de privacidad actualizado (capítulo 7).

¿Gemini entrena con los datos de los estudiantes? No, si la clave es de un proyecto de Google Cloud con facturación activa (servicios de pago de la API de Gemini): en ese caso Google no usa los *prompts* ni las respuestas para mejorar sus productos, y solo los registra por tiempo limitado para detectar abuso y por requisitos legales. En el nivel gratuito sí los usa para mejorar productos, revisores humanos pueden leerlos y los términos piden no enviar información personal; por eso **nunca se usa el nivel gratuito con estudiantes reales**. Fuente: `ai.google.dev/gemini-api/terms` . Capítulo 7.8.

¿Qué pasa si Gemini no responde? Responde el mentor local, y el estudiante lo ve. Si el servidor está apagado o no tiene clave, el pie del mentor dice *"Mentor local..."* desde que se abre. Si el servidor tenía Gemini pero una respuesta falla, esa respuesta lleva la nota *"Gemini no pudo responder en este momento, así que respondió el mentor local"*. El backend devuelve códigos claros para cada falla (`503 mentor_not_configured` , `502 mentor_key_rejected` , `502 mentor_upstream` , `502 mentor_blocked` , `504 mentor_timeout` , `429 mentor_rate_limited`) y la extensión cae al mentor local con cualquiera de ellos. El estudiante siempre recibe respuesta; su progreso no depende de Google.

¿FARO guarda las conversaciones con el mentor? No. Viven en la memoria de la extensión durante la sesión. El backend no las guarda ni las registra: ni el mensaje, ni la respuesta, ni el contexto.

Vigilancia y bienestar

¿FARO calcula un "riesgo de abandono" del estudiante? Calcula estados de fricción internos (cinco etiquetas basadas en días sin actividad y trabajo pendiente) para decidir qué intervención ofrecer. **Nunca** muestra un puntaje de riesgo al estudiante y no lo comparte con nadie. La interfaz dice "tu ruta necesita atención", no "riesgo: 87%".

¿Puede un docente ver quién estudia cuánto? No existe esa pantalla, y el modelo de datos de Comunidad solo admite conteos agregados, no personas. Si la institución quisiera métricas del piloto, se propone entregarlas agregadas y con ids pseudónimos.

¿FARO envía notificaciones o recordatorios? No. No hay notificaciones push, correos ni recordatorios. FARO espera a que el estudiante abra el panel. El "puerto seguro" (pausa planeada) explícitamente no tiene cuenta regresiva.

¿Es FARO un videojuego? No. Usa mecánicas (ruta, puntos de control, logros) sin vidas, niveles ni tablas de posiciones. Los cinco logros y cinco reconocimientos exigen haber persistido; ninguno se gana haciendo clic.

Operación

¿Dónde corre el backend? En el piloto, en una máquina del equipo o de la institución, escuchando en `127.0.0.1`. En producción, en infraestructura de la institución o en un proveedor aprobado, detrás de TLS, con el lanzamiento LTI como autenticación.

¿Qué dependencias tiene? El backend, ninguna. Solo Node.js. La extensión, dos: `react` y `react-dom`.

¿Cómo sé que las garantías de este libro son ciertas? Ejecutando `npm test` dentro de `server/`: treinta y ocho verificaciones automáticas contra un Canvas simulado con paginación real y un Gemini simulado. Y abriendo *Perfil* → *Canvas* en FARO: el registro de peticiones en vivo muestra cada llamada.

¿Qué falta para producción? Registro LTI 1.3 por la institución, OAuth2 por estudiante, base de datos con cifrado en reposo, validar el mentor con Gemini contra la API real con una clave de pago de la institución, aviso de privacidad formal y revisión de seguridad externa. Capítulo 9 tiene la lista completa.

¿FARO reemplaza a Canvas? No. Canvas sigue siendo la fuente de verdad académica. FARO lee de Canvas y añade una capa de acompañamiento. *No reemplazamos Canvas: lo hacemos adaptativo.*

9. Lista de verificación antes de un piloto

Marcar cada punto antes de conectar FARO a un Canvas con estudiantes reales. Los puntos con ⚙ los verifica el equipo técnico; los puntos con 🏛 los verifica la institución.

A. Antes de arrancar el backend

- ☐ ⚙ `server/.env` existe, **no** está en git (`git status` no lo lista) y `CANVAS_API_URL` empieza con `https://`.
- ☐ ⚙ El token pertenece a una **cuenta de estudiante** del piloto, no a un docente ni administrador.
- ☐ ⚙ El token tiene **fecha de expiración** (recomendado: 30 días).
- ☐ ⚙ `FARO_HOST=127.0.0.1` salvo que haya TLS por delante.
- ☐ ⚙ `FARO_ALLOWED_ORIGINS` contiene solo los orígenes reales (el id exacto de la extensión cargada, el servidor de desarrollo si aplica).
- ☐ ⚙ `FARO_STRICT_ORIGINS=true` en cualquier despliegue que no sea la máquina de un desarrollador: así el backend deja de aceptar cualquier origen `chrome-extension://` (capítulo 3.6).
- ☐ ⚙ Si se usará el mentor con Gemini: `GEMINI_API_KEY` es de un proyecto de Google Cloud **con facturación activa** (servicios de pago), **nunca** del nivel gratuito (capítulo 7.8).
- ☐ ⚙ `GEMINI_MODEL` se verificó contra `ai.google.dev/gemini-api/docs/models` (por omisión `gemini-3.5-flash`; los nombres de modelo cambian).
- ☐ ⚙ `FARO_MENTOR_PER_MINUTE` tiene un valor acordado (por omisión 20 llamadas al modelo por cliente y minuto).
- ☐ ⚙ `npm test` en `server/` pasa las 38 verificaciones.
- ☐ ⚙ `curl http://127.0.0.1:3000/health` responde con el host de Canvas y `mentor: {provider, model}` (o `null` si no hay clave), y **no** contiene el token ni la clave.
- ☐ ⚙ El mentor con Gemini se probó contra la API real con la clave del piloto: una respuesta llega con el anillo violeta y **sin** la nota "Gemini no pudo responder...". (En desarrollo solo se verificó contra un Gemini simulado.)

B. Antes de cargar la extensión

- ☐ ⚙ Se construyó con `VITE_CANVAS_MODE=http` y `VITE_FARO_API_URL` apuntando al backend.
- ☐ ⚙ `VITE_MENTOR_MODE=local` salvo que la institución haya aprobado el mentor con IA; en ese caso, `auto` (por omisión) o `gemini`, con una clave de pago aprobada en el servidor. El pie de FARO Mentor dice lo correcto ("Responde Gemini, a través del servidor de FARO..." o "Mentor local: sin modelo de IA...").
- ☐ ⚙ Ni `.env` ni `.env.local` de la raíz contienen una clave de Gemini.
- ☐ ⚙ `public/manifest.json` **no** contiene `instructure.com` en `host_permissions`.
- ☐ ⚙ En *Perfil* → *Canvas* la insignia dice *En vivo* y el registro muestra estado `200` en las cuatro rutas.
- ☐ ⚙ Si aparece la nota "tu cuenta no puede leer las analíticas", se decidió con la institución si se habilita el permiso o se acepta el respaldo por entregas.

C. Con la institución

- ☐  Se confirmó qué ley aplica (LFPDPPP para institución privada; Ley General para pública).
- ☐  Existe un aviso de privacidad (simplificado + completo) revisado por jurídico, que menciona: las seis rutas de Canvas, los datos que FARO crea, el estado "pasó algo en mi vida", el mentor con Gemini si se activa (qué se envía a Google, capítulo 7.6) y la retención.
- ☐  Cada participante firmó el consentimiento del piloto (capítulo 7.4) y es mayor de 18 años (requisito de los términos de la API de Gemini).
- ☐  Se acordó cómo se ejercen los derechos ARCO durante el piloto y quién responde.
- ☐  Se acordó qué métricas del piloto se entregan y en qué forma (agregadas, con ids pseudónimos).
- ☐  El área de seguridad de la institución revisó este libro y el código de `server/`.
- ☐  Se definió la fecha de fin del piloto y el procedimiento de revocación de tokens y borrado de datos al terminar.

D. Al terminar el piloto

- ☐  Todos los tokens del piloto revocados en Canvas.
- ☐  La clave de Gemini del piloto revocada o rotada en Google.
- ☐  `server/.env` borrado de la máquina del backend.
- ☐  Participantes informados de cómo borrar el estado local (*Progreso* → *Restablecer* o desinstalar).
- ☐  Métricas entregadas; datos crudos destruidos o entregados a la institución según lo acordado.

E. Para pasar a producción (fuera del alcance del piloto)

- ☐ Registro de FARO como herramienta LTI 1.3 a nivel de cuenta en Canvas, con los scopes exactos de `requiredScopes` y sin `score` ni `lineitem`.
- ☐ Implementación de `/lti/login`, `/lti/launch` y `/.well-known/jwks.json` según la especificación en `server/src/routes/lti.ts`.
- ☐ OAuth2 por estudiante; eliminación del token compartido.
- ☐ PostgreSQL con cifrado en reposo; política de retención automática.
- ☐ Revisión jurídica de los términos vigentes de la API de Gemini (o del proveedor que se elija) y del contrato de la institución con Google.
- ☐ Revisión de seguridad externa.